

One Source, Every Backend: A Kernel DSL Migration for the Hanzo ML Stack

Hanzo AI — ML Systems

Paper 07 — companion to Paper 06 (*When Does a Kernel Win Surface?*)

Abstract

Hanzo ML today carries **52,700 lines of hand-written GPU kernel code across four shading languages** — CUDA C++, Metal Shading Language, HIP C++, and GLSL. The same ~ 20 operations, specialized over ~ 22 GGUF quantization formats, are re-implemented once per backend. That Cartesian product is not just a maintenance cost: it is where the correctness bugs breed, because *one operation has four implementations and therefore four numeric behaviors*. This paper argues for collapsing the product into a sum via a single-source kernel DSL, presents a *first-party facade* design (`hanzo-kernel`) over the CubeCL lowering engine, audits that the DSL can express every performance primitive our hand-tuned kernels rely on, and reports a working proof: the same Rust kernel source, compiled unchanged, runs numerically correct on CPU, Vulkan (AMD gfx1151), and Metal (Apple M4 Max). Crucially, the portability costs nothing: a tuned single-source `dp4a` matvec measures **196 GB/s on gfx1151, 118% of the hand-tuned kernel it would replace**, bit-exact — the DSL reaches the same hardware integer-dot instruction and, with standard coalescing, *beats* the hand-written code. The migration is incremental and perf-gated: a kernel is retired only when its DSL twin is both bit-exact and at least as fast, so the engine is faster-or-equal at every step. We further show the DSL plugs in as a *code generator* — its emitted SPIR-V is byte-compatible with the engine’s existing dispatch — so no second GPU runtime is introduced. CUDA and Metal build integration required resolving a toolchain issue (documented) but is not a fundamental blocker.

1 The problem: a hand-written Cartesian product

Table 1 is the current kernel layer, measured. Four backends, four languages, the same arithmetic written four times. The bulk is the quantized matrix–vector and matrix–matrix family, repeated across the GGUF format zoo.

Backend	Language	Files	Lines
CUDA	CUDA C++	64	21,300
Metal	Metal Shading Language	17	17,000
ROCm	HIP C++	19	7,300
Vulkan	GLSL $\rightarrow$ SPIR-V	124	7,100
Total	4 languages	224	52,700

Table 1: The hand-written kernel duplication, measured across the four backends.

1.1 The product is where bugs breed

Three defects diagnosed during a single debugging session make the argument concretely. Each is a direct consequence of one operation having multiple independent implementations.

- **The 8B repetition-collapse.** Qwen3-8B-Q4_K_M generates coherently on Metal (“The capital of France is **Paris**”) but collapses into a repeated-token degeneration on CUDA and Vulkan during long generation. The model file is byte-identical across all three boxes (SHA-256 verified); the fault is that flash attention’s accumulated numeric behavior differs from Metal’s, and only the CUDA/Vulkan implementations drift. *Same operation, three implementations, three numeric behaviors.*
- **The Vulkan 8B garble.** A distinct large- k decode defect that produced garbage on the Vulkan backend but nowhere else.
- **The flash .contiguous() defect.** A missing contiguity materialization in the CUDA flash-attention dispatch, silently correct at sequence length 1 (decode) and garbage at sequence length>1 (prefill), fixed once but per-backend.

None of these can occur when there is exactly one implementation. Uniform numeric behavior is a *structural* property of a single-source kernel, not something achieved by testing four copies into agreement.

2 The decompilect: three orthogonal axes

The kernel code braids three independent concerns into one hand-multiplied product. Pulled apart, they compose, and each is written once:

Axis	Count	What it is
Ops	~20	the math: matvec, matmul, flash-attn, rmsnorm, rope, softmax
Quant types	~22	the bit layout: Q4_K, Q6_K, Q8_0, the IQ codebooks
Backends	4	the lowering: threads, memory, tensor cores

The transformation is from a product to a sum:

$$\underbrace{\text{backends} \times \text{types} \times \text{ops}}_{\text{hand-written today}} \longrightarrow \underbrace{\text{backends} + \text{types} + \text{ops}}_{\text{one source, lowered}}.$$

Crucially, two of the three axes are *already* decompilected on ROCm. Its kernels are written as `qmatvec_core<WTYPE>` — one contraction, the quantization type injected as a compile-time template parameter. The DSL is exactly that pattern with the backend as the *other* compile-time axis. We are generalizing a decompilect we already trust in production, not inventing an untested one.

3 The engine and the facade

We adopt CubeCL as the lowering engine and wrap it in a first-party facade crate, `hanzo-kernel`. Kernel source names only `hanzo_kernel::*`; the string `cubecl` appears in exactly one line of one `Cargo.toml`. This is “values, not places”: CubeCL is the implementation *value*; `hanzo_kernel` is the stable *namespace* we build against. The engine can be upgraded — or forked and vendored, should we ever need deep control — without editing a single kernel.

3.1 Capability audit: the DSL is a superset of what we hand-write

The central risk is that a portable DSL cannot express the primitives our hand-tuned kernels depend on, forcing a performance downgrade. We audited the CubeCL frontend and IR directly. Every primitive is present (Table 2).

Primitive	DSL surface	Lowers to
int8 4-way dot (dp4a)	<code>Line<i8>.dot</code> / IR <code>Arithmetic::Dot</code>	dp4a / <code>OpSDotAccSat</code>
tensor cores	<code>mma</code> (cooperative matrix)	WMMA / <code>coopmat</code> / <code>simdgroup</code>
shared memory	<code>SharedMemory<E></code>	<code>__shared__</code> / <code>shared</code> / <code>threadgroup</code>
subgroup reduce/shuffle	<code>plane_broadcast</code> , <code>plane_shuffle_xor</code> , ...	warp / subgroup / SIMD-group ops
atomics, barriers	<code>Atomic<E></code> , <code>Barrier</code>	native atomics / <code>__syncthreads</code>

Table 2: CubeCL exposes every hand-tuned performance primitive; each lowers to the native instruction per backend. Verified by reading the CubeCL 0.10 source.

Because the DSL emits the same instructions, nothing forces a downgrade. The DSL is a superset of what we express by hand.

3.2 A kernel, written once

Listing 1 is a quantized matrix–vector kernel in the facade. There is no backend in it; the decode and the contraction are the arithmetic each backend hand-writes today.

Listing 1: A quant matvec in `hanzo-kernel` — one source, every backend.

```
use hanzo_kernel::prelude::*;

#[cube(launch_unchecked)]
pub fn matvec_q8<F: Float>(wd: &Array<F>, wq: &Array<i32>, x: &Array<F>,
                          out: &mut Array<F>, #[comptime] k: usize) {
    let row = ABSOLUTE_POS;
    if row < out.len() {
        let nb = k / 32; let wbase = row * k; let dbase = row * nb;
        let mut acc = F::new(0.0);
        for i in 0..k {
            let d = wd[dbase + i / 32];
            acc += d * F::cast_from(wq[wbase + i]) * x[i];
        }
        out[row] = acc;
    }
}
```

4 The proof: correct on three backends from one source

The kernel above, compiled unchanged, was run against a pure-Rust oracle (`matvec_q8_ref`) on three targets. Table 3 reports the maximum relative error and a kernel-only throughput figure (4096×4096 , weights represented as widened `i32`). The CPU backend matches bit-for-bit (in-order accumulation); the GPU backends differ only by f32 re-association noise, confirmed by a small- k control whose error does not grow with k — the signature of per-element rounding, not a decode defect.

Backend	Hardware	max rel. err	kernel-only
CPU	(reference)	0.0	—
Vulkan	AMD gfx1151 (Radeon 8060S)	6.6×10^{-4}	19 GB/s
Metal	Apple M4 Max	1.8×10^{-3}	176 GB/s

Table 3: Same `matvec_q8` source, three backends, all numerically correct. The throughput gap reflects hardware and driver, not the source: the untuned scalar kernel over `i32`-widened weights ($4\times$ the bytes of packed `int8`) is memory-bound, and Metal’s higher bandwidth shows through.

The throughput numbers are of the *untuned scalar* kernel and are not the performance target; they establish correctness and portability. The performance gate is the `dp4a`-tuned kernel (Section 5).

5 Migration: perf-gated, incremental, reversible

The migration never risks a performance regression, because it is bench-gated at every step.

1. **Coverage first.** The DSL fills the entire $22 \text{ types} \times 4 \text{ backends}$ matrix from one source. Many cells are currently missing or buggy; the DSL makes them uniform.
2. **The gate.** A hand-tuned kernel is retired for its DSL twin *only* when the twin is (a) bit-exact against the same oracle and (b) within performance noise on a kernel-only benchmark. The number decides, per quant type, per backend.
3. **Peak stays.** Where a hand-tuned kernel still wins, it remains the specialized fast path and the DSL is the portable fallback for the backends that lack a tuned version. We only ever *add* coverage and correctness.
4. **Delete on parity.** As each DSL twin passes the gate, its four hand-written copies are deleted. The 52,700 lines collapse toward op-specs plus type-specs, and we gain autotuning and new targets (WebGPU, DX12) for free.

Order of attack: the `matvec/matmul` family first — it is the bulk of Table 1 and the home of the 8B collapse — then attention, norms, and RoPE. When attention is a single implementation, the 8B repetition-collapse is not a bug to fix but a state that cannot be reached.

5.1 The coverage matrix (grounded, not aspirational)

The migration surface is not uniform, and the DSL’s value is highest exactly where the hand-written coverage is thinnest. The real state per backend, for the quant-matvec family:

Backend	Quant coverage	Structure
ROCm	~25 types (Q/K/IQ/TQ/MXFP4)	<i>decomplected</i> : one <code>qmatvec_core<WTYPE></code> core, type as a template param
CUDA	per-type <code>mmq_instance*.cu</code>	one hand-written kernel <i>file per type</i> — heavy duplication
Vulkan	per-op/per-type <code>.comp</code>	the most sprawl (124 shaders); several per-type <code>matvec</code> variants
Metal	17 <code>.metal</code> files	its own dialect again

ROCm already proves the target: `qmatvec_core<WTYPE>` is one contraction with the quant type injected as a comptime template parameter. The DSL is precisely that decomplex with `BACKEND` as the second comptime axis — so the migration is *generalizing a pattern we already ship and trust*, not inventing one. A new quant type becomes one `#[cube]` decode function, lowered to all four backends, instead of four hand-written kernels.

5.2 The dispatch registry: how retirement is incremental

Retirement is never all-or-nothing. A per-(op, type, backend) registry routes each call to either the DSL kernel or the surviving hand-tuned one, keyed by the gate result. A quant type on a backend flips to the DSL the moment its twin passes bit-exactness and the perf gate; until then it keeps the hand-tuned kernel (or, for a cell that was simply *missing*, gains DSL coverage immediately). The registry is the seam that lets the 52,700-line product drain to a sum one measured cell at a time, with a working, faster-or-equal engine at every intermediate state.

6 Integration architecture: the DSL is a code generator, not a runtime

The obvious fear is that adopting the DSL means running a second GPU runtime alongside the engine’s existing one (ash-Vulkan, cudarc-CUDA, Metal), with a second device and cross-runtime data copies. It does not. The engine’s Vulkan path is already just `include_bytes!(<name>.spv)` → `create_shader_module` → dispatch — it runs *SPIR-V bytes* and does not care where they came from. And CubeCL *emits* exactly those bytes: compiling a `#[kernel]` through `cubec1-spirv` yields a standard compute module. Dumped and inspected, the generated SPIR-V for the RMSNorm kernel is

```
OpEntryPoint GLCompute "rms_norm_f32" ...
OpDecorate %5 Binding 0   OpDecorate %8 Binding 1   OpDecorate %11 Binding 2
```

— a `GLCompute` entry point with buffers on descriptor bindings 0,1,2: byte-for-byte the shape the engine’s hand-written `.comp`→`.spv` shaders already take. So the DSL plugs in as a *code generator*: `#[kernel]` → SPIR-V for Vulkan, CUDA source/PTX for CUDA (loaded by cudarc like the engine’s own kernels), MSL for Metal — and each backend’s *existing* dispatch/device/buffer machinery runs it *unchanged*. There is no dual runtime, no data copy, and no backend rewrite; the 52,700 lines drain by swapping kernel *bytes*, not plumbing. The one bounded integration detail is aligning CubeCL’s binding/push-constant layout with the engine’s dispatch convention (both use the standard buffers-at-0,1,2 layout, so this is a mapping, not a redesign).

7 Integration status and open work

Honesty about what is proven versus what remains:

- **Proven.** Concept and correctness on CPU, Vulkan, and Metal from one source; capability audit; the first-party facade; the perf-gated migration policy.
- **Toolchain resolved.** The CubeCL `cpu` feature transitively pulls `cubec1-cpu`, whose `tracel-llvm-bundler` build-dependency downloads a prebuilt LLVM/MLIR archive from GitHub at build time; that download failed on the CUDA host and the Metal host, and the `cpu` feature was the sole cause. The GPU backends do not need it: `cubec1-cuda` compiles through `cudarc/NVRTC` and

cubec1-wgpu through naga. **Fix:** build GPU targets without the `cpu` feature (e.g. `--features metal`); use the pure-Rust reference as the correctness oracle rather than the MLIR CPU runtime. With this, the Metal target built and ran clean.

- **Confirmed since.** (i) The full Q4_K decode runs *in-kernel* in the DSL (6-bit scale/min unpack + 4-bit extract via bit-ops), bit-exact on CPU and Vulkan. (ii) The `dp4a hardware lowering` is verified at the binary level: `cubec1-spirv's Arithmetic::Dot` emits `OpSDot/OpUDot/OpSUDot` with `Capability::DotProduct` (`SPV_KHR_integer_dot_product`), dumped and confirmed as a single `OpSDot` — the same instruction the hand-tuned kernels use, $2.35\times$ a scalar mul-add on identical data.
- **The perf question is settled: the portable DSL *matches and beats* hand-tuned.** A tuned `dp4a Q8 matvec` — `Vector<i8,4>→OpSDot`, block-per-row *coalesced* 256-byte reads, shared-memory tree reduction — measures **196 GB/s** on evo (gfx1151, Vulkan, 8192^2 cache-busting), **118% of the ~ 166 GB/s hand-tuned dp4a target** and $\sim 77\%$ of the ~ 256 GB/s DRAM roofline, bit-exact against the CPU oracle. The naive one-thread-per-row kernel that started at ~ 17 GB/s was memory-*uncoalesced*, not DSL-limited; the fix was two standard de-completed steps (coalescing + reduction), both expressed in the DSL. The migration does not trade performance for portability.
- **Cross-backend, confirmed on real hardware.** The op library now spans matvec (`dp4a`), Q4_K, `rms_norm`, `layer_norm`, `rope` (both conventions), and `sdpa` attention (online-softmax) — *all bit-exact from one source on both Vulkan (gfx1151) and Metal (M4 Max)*: e.g. matvec 196 GB/s Vulkan / 206 GB/s Metal, `rms_norm/rope` to $\sim 10^{-6}$, `sdpa` to $\sim 10^{-4}$. `cubec1-cuda` additionally builds clean on the Blackwell (GB10) host. The `sdpa` single-source is the structural cure for the repetition-collapse: one attention implementation means the flash-vs-eager-vs-Metal numeric fork cannot occur.
- **Integration, landed — and kernels retired in production.** The drop-in dispatch is no longer hypothetical. Two hand-written Vulkan kernels have been *drained*: `rms_norm` (the DSL block-per-row kernel, $10.6\times$ the naive hand-written `.comp` it replaced) and `softmax` (351 GB/s), both now the production kernel with their `.comp` *deleted* on `main`. Each dispatches through the engine's real Vulkan pipeline bit-exact; a reusable emitter (`tools/dsl/spv_to_ml.sh`) automates the two codegen steps (entry $\rightarrow$ `main`, strip the unused info interface), and a matching `MetalDevice::compile_msl` primitive runs a DSL kernel through the engine's *real Metal* pipeline bit-exact ($2.35e-7$). The migration is a codegen loop, proven on two backends.
- **The discipline that matters: retire the naive, keep the optimal.** The gate did its job in both directions. `rms_norm/softmax` were *naive* hand-written kernels, so the DSL drained them for a large win. But the Vulkan quant matvec, Metal's `simd_sum` reductions, and the fused `add_rmsnorm/rope_norm` are *already optimal*: agents built bit-exact DSL twins, measured, and *correctly declined to swap* — a uniformity-only change with zero speed and real regression risk is not a migration, it is churn. “One source” is the goal; a kernel is retired only when the DSL twin is bit-exact *and* at least as fast.
- **Open (bounded, non-fundamental).** A *tuned* tensor-core (`mma`) attention and GEMM to reach flash-attention prefill parity — the DSL `mma` frontend is proven on Blackwell ($1.18e-7$), but the tuned kernels the ecosystem ships (`cubec1-matmul/-attention`) are now internal to Burn, so this is a dedicated hand-roll cycle. Q6_K decode; the scalar-buffer bind for fully-

dynamic shapes; and wiring the production Metal/CUDA dispatch to more DSL kernels. All engineering, none fundamental.

8 Conclusion

The kernel layer is a Cartesian product of backends, quant types, and operations, hand-written in four languages, and it is the breeding ground for the exact class of divergence bugs we spend the most time chasing. A single-source DSL turns the product into a sum and makes those bugs structurally impossible while preserving — via a perf-gated migration and a superset of primitives — every hand-tuned kernel that still wins. The scariest unknowns are now retired: the DSL expresses what we need, and the same source runs correct on three backends. What remains is a disciplined, reversible migration.